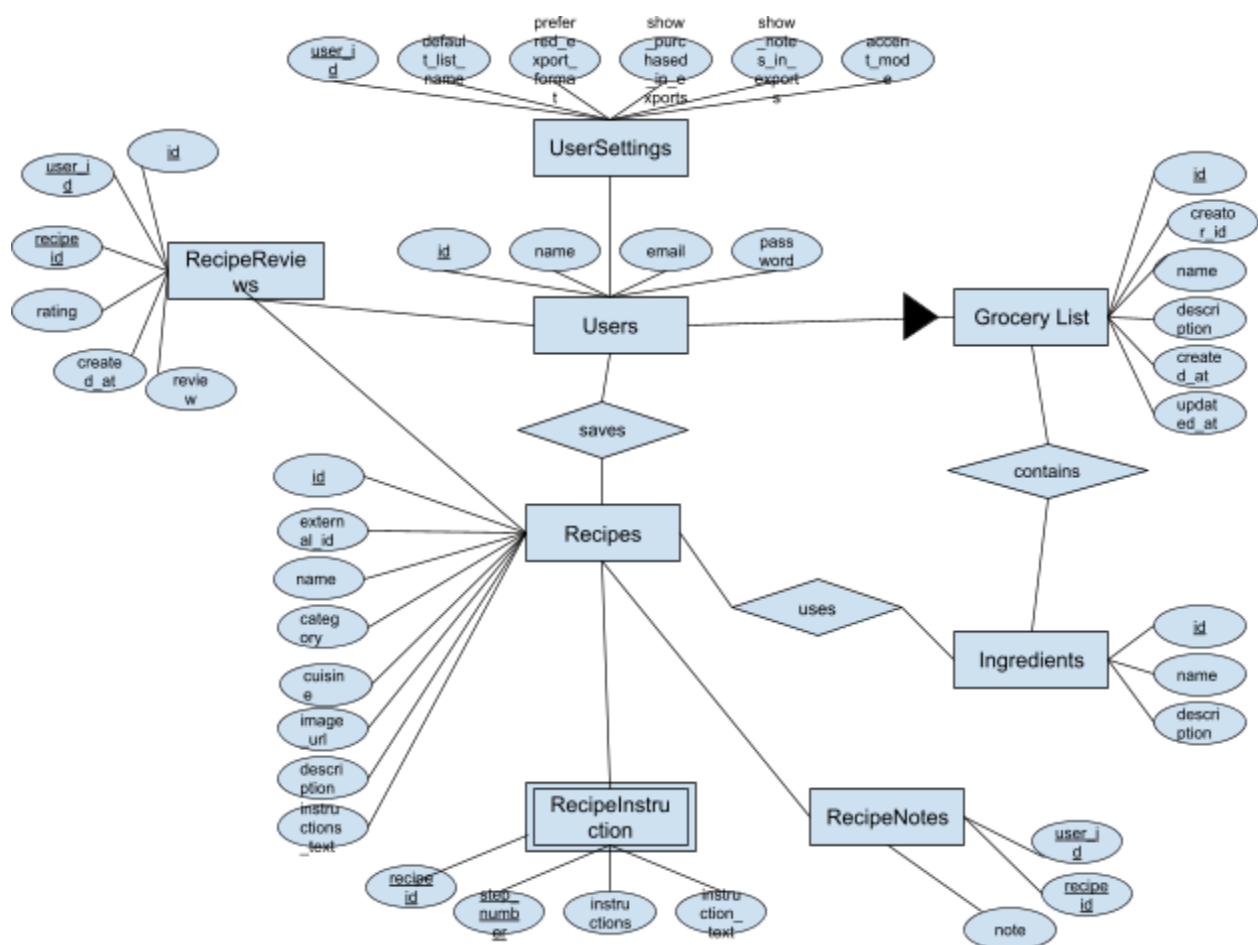


Sia Savkoor - pyk5ux
Suzu Paudel - uwz3vh
Neeha Mikkilineni - gfv5mu

Extra Credit: We received help from Laxmi Ghanate (bnk2zk), Sahiti Palleti (hfd8vv), Max Grant (gbe9wk). Laxmi helped us when we were facing conflicting schema issues, and she helped us figure out how to consolidate our previous schema with our current schema and queries. Sahiti helped us resolve an issue with our database level security by explaining how to use certain commands for access control. Max helped us with application level security, specifically password hashing and token authentication. We helped them with Google Cloud issues. We both learned a lot from collaborating with each other.

1. Database design

- Include a final version of your E-R diagram



- Include a final version of your tables written in schema statements

users(id, name, email, password, role)

recipes(id, external_id, creator_id, name, category, cuisine, image_url, description, instructions_text)

ingredients(id, name, description)

recipe_instructions(recipe_id, step_number, instructions, instruction_text)

uses(recipe_id, ingredient_id, quantity, unit)

grocery_lists(id, creator_id, name, description, created_at, updated_at)

contains(grocery_list_id, ingredient_id, quantity, unit, purchased, note)

recipe_reviews(id, recipe_id, user_id, rating, review, created_at)

saves(user_id, recipe_id, is_favorite, saved_at)

recipe_notes(user_id, recipe_id, note)

user_settings(user_id, default_list_name, preferred_export_format, show_purchased_in_exports, show_notes_in_exports, accent_mode)

2. Database programming

- Specify where you host your database

If you do the extra credits (hosting your database and your app on GCP), submit screenshot(s) showing your database on GCP

We host our database on Google Cloud Platform using Cloud SQL for MySQL. Our PantryPal MySQL database is stored in a Cloud SQL instance, which contains the tables for users, recipes, ingredients, saved recipes, grocery lists, recipe instructions, recipe reviews, recipe notes, and user settings.

Google Cloud | pantrypal | Search (/) for resources, docs, products, and more

Databases

All instances > pantrypal-db

pantrypal-db
MySQL 8.0

i Instance is being updated. This may take a few minutes. While this operation is running, you may continue to view information about the instance.

+ Create database

Name ↑	Collation	Character set	Type	
information_schema	utf8mb3_general_ci	utf8mb3	System	⋮
mysql	utf8mb3_general_ci	utf8mb3	System	⋮
pantrypal	utf8mb4_0900_ai_ci	utf8mb4	User	⋮
performance_schema	utf8mb4_0900_ai_ci	utf8mb4	System	⋮
sys	utf8mb4_0900_ai_ci	utf8mb4	System	⋮

Uploads

- Specify where you host your app — the deployment environment needed to deploy and run your project
 - If you do the extra credits (hosting your database and your app on GCP), submit the URL to your app

The PantryPal app is hosted in our GitHub repository and run locally for the final demonstration. The app runs in a local Node.js/npm environment with a React/Vite frontend and an Express/Node backend. The backend connects to our Google Cloud SQL MySQL database using private environment variables in a .env file. To run the project, a user clones the repository, installs dependencies, configures the database environment variables, and starts the frontend and backend with npm.

- Describe and include all instructions / steps needed to deploy and run your project

Note: we may randomly set up, deploy, and run your project on a similar environment. It is important that you provide the description and a complete set of the instructions.

Clone the github repository using `git clone https://github.com/psuzu/PantryPal.git`. Then use the command `cd PantryPal`. From the project root, first install the root helper package by running `npm install`. Once that completes, install all app dependencies with `npm run install:all`. Configure the backend environment file by creating `.env` in the server directory. Add the following:

```
DB_HOST=35.245.162.21
DB_USER=pantrypal_user
DB_PASSWORD=NewStrongPassword123!
DB_NAME=pantrypal
DB_PORT=3306
PORT=3001
```

After everything is set up, start both the frontend and backend together using `npm run dev`. When the development servers are running, open the Vite URL displayed in your terminal to view the application. Use `ctrl c` to terminate. When creating the account, the password has to be more than 8 characters long.

3.
 - Refer to advanced SQL commands you wrote in the previous milestone, clearly discuss how the advanced SQL commands are incorporated in your app, what features of your app use them, and how they reflect the database

We used the following advanced SQL commands in our app:

```
1.
CREATE PROCEDURE GetUserRecipes(IN userId INT)
BEGIN
  SELECT * FROM recipes
  WHERE creator_id = userId;
END;
```

The stored procedure `GetUserRecipes` retrieves recipes connected to a specific user. This procedure selected recipes where `creator_id` matched the given user ID. In the final app, this logic relates to the user-specific recipe features, especially the saved/owned recipe views. It reflects the database because recipes are tied back to users through user-specific relationships, either through `creator_id` for created recipes or through the `saves` table for saved recipes. This supports the idea that each user should only see recipe data connected to their own account.

2.

```
CREATE TRIGGER before_update_review_rating
BEFORE UPDATE ON recipe_reviews
FOR EACH ROW
BEGIN
  IF NEW.rating < 1 OR NEW.rating > 5 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Rating must be between 1 and 5';
  END IF;
END;
```

The trigger `before_update_review_rating` is tied directly to the `recipe_reviews` table. It runs before a review rating is updated and prevents the database from accepting a rating outside the valid 1–5 range. This supports the recipe review/rating feature of the app because recipe ratings are stored in `recipe_reviews` and are used when calculating average recipe ratings. The app can display rating information for recipes, but the database is responsible for enforcing that all stored ratings remain valid.

These advanced SQL commands reflect the structure of our database because they use relationships between users, recipes, and `recipe_reviews`. The stored procedure supports user-specific recipe retrieval, while the trigger enforces an integrity rule on review data. They show that PantryPal uses SQL for database-level business logic and data validation, not just basic insert, update, and delete operations.

4. Describe the database security at the database level

- Specify whether the security is set for developers or end users
- Discuss how you set up security at the database level (access control)
- Submit the SQL commands you use to limited / set privileges such as

```
GRANT privilege-name ON object-name TO {user-name | PUBLIC
| role-name} or
```

```
REVOKE privilege-name ON object-name FROM {user-name |
PUBLIC | role-name}
```

Our database security is set for developers and the application backend, not individual end users. End users access data through the PantryPal app, while the backend connects to the database using a controlled database account. At the database level, we used Role-Based Access Control (RBAC). We created two roles: ``app_user`` and

`admin\_user`. The `app\_user` role is used by the application backend and only has the privileges needed for normal app functionality, such as reading recipes, saving recipes, and creating or updating grocery lists. The `admin\_user` role is for developers or database administrators and has full privileges for database management tasks. This matches RBAC because permissions are assigned to roles, and database users are granted those roles.

```
CREATE ROLE IF NOT EXISTS 'app_user';
CREATE ROLE IF NOT EXISTS 'admin_user';
CREATE USER IF NOT EXISTS 'app_client'@'%' IDENTIFIED BY 'strongpassword';
CREATE USER IF NOT EXISTS 'admin_client'@'%' IDENTIFIED BY 'strongpassword';
GRANT 'app_user' TO 'app_client'@'%';
GRANT 'admin_user' TO 'admin_client'@'%';
SET DEFAULT ROLE 'app_user' TO 'app_client'@'%';
SET DEFAULT ROLE 'admin_user' TO 'admin_client'@'%';
GRANT SELECT, INSERT, UPDATE, DELETE ON pantrypal.recipes TO 'app_user';
GRANT SELECT ON pantrypal.ingredients TO 'app_user';
GRANT SELECT, INSERT, UPDATE, DELETE ON pantrypal.grocery_lists TO
'app_user';
GRANT SELECT, INSERT, UPDATE, DELETE ON pantrypal.saves TO 'app_user';
GRANT ALL PRIVILEGES ON pantrypal.* TO 'admin_user';
```

5. Describe the database security at the application level

- Discuss how database security at the application level is incorporated in your project.
- Submit code snippet(s) to illustrate how security aspect is implemented and to support your discussion.

Database security in PantryPal is enforced at the application level by ensuring that all database interactions are controlled and validated by the backend rather than trusting client input. First, user passwords are never stored in plaintext. Instead, they are securely hashed using the Node.js `crypto.scrypt` algorithm with a random salt before being inserted into the database, and verified during login using a safe comparison method. Second, the application replaces the insecure practice of trusting a client-provided user ID with a signed token-based authentication system. When a user logs in or signs up, the server generates a signed token using HMAC, which the client must include in future requests; this token is verified on every protected route to confirm the user's identity.

```

export function validatePassword(password) {
  if (typeof password !== 'string' || password.length < 8) {
    throw new Error('Password must be at least 8 characters long.')
  }
}

export function isPasswordHash(value) {
  return typeof value === 'string' && value.startsWith(`${PASSWORD_HASH_PREFIX}$`)
}

export async function hashPassword(password) {
  validatePassword(password)

  const salt = crypto.randomBytes(16).toString('hex')
  const derivedKey = await scryptAsync(password, salt, PASSWORD_KEY_LENGTH)

  return `${PASSWORD_HASH_PREFIX}${salt}${Buffer.from(derivedKey).toString('hex')}`
}

export async function verifyPassword(password, storedPassword) {
  if (typeof password !== 'string' || typeof storedPassword !== 'string') {
    return false
  }

  if (!isPasswordHash(storedPassword)) {
    return safeCompare(password, storedPassword)
  }

  const [, salt, expectedHash] = storedPassword.split('$')
  if (!salt || !expectedHash) {
    return false
  }
}

```

```

export function createAuthToken(user) {
  const payload = {
    sub: Number(user.id),
    exp: Date.now() + TOKEN_TTL_MS,
  }
  const encodedPayload = encodeBase64Url(JSON.stringify(payload))
  const signature = crypto
    .createHmac('sha256', tokenSecret)
    .update(encodedPayload)
    .digest('base64url')

  return `${encodedPayload}.${signature}`
}

export function verifyAuthToken(token) {
  const [encodedPayload, signature] = String(token || '').split('.')
  if (!encodedPayload || !signature) {
    throw new Error('Invalid token format.')
  }

  const expectedSignature = crypto
    .createHmac('sha256', tokenSecret)
    .update(encodedPayload)
    .digest('base64url')

  if (!safeCompare(signature, expectedSignature)) {
    throw new Error('Invalid token signature.')
  }

  const payload = JSON.parse(decodeBase64Url(encodedPayload))
  if (!Number.isInteger(payload.sub) || payload.exp <= Date.now()) {
    throw new Error('Token expired or malformed.')
  }

  return { id: payload.sub }
}

```